

Dokumentacja Techniczna Systemu Zarządzania Kliniką

Architektura, API oraz Wymagania Funkcjonalne

Jakub Peska

30 maja 2026

Spis treści

1	Wprowadzenie	2
1.1	Cel dokumentu	2
1.2	Założenia architektoniczne	2
2	Wymagania Funkcjonalne	3
2.1	Wstęp	3
2.2	Przypadki Użycia i Zachowanie Systemu	3
2.2.1	Zarządzanie Wizytami i Smart Calendar	3
2.2.2	Zarządzanie Gabinetami (Offices / Rooms)	3
2.2.3	Zarządzanie Lekarzami i Pacjentami	4
2.2.4	System Audytu (Blackbox) i Inicjalizacja	4
3	Diagram Bazy Danych	5
3.1	Wstęp	5
3.2	Schemat ERD (Entity-Relationship Diagram)	5
4	Specyfikacja API Backendu	7
4.1	Wstęp	7
4.2	Kluczowe Endpointy	7
4.2.1	Tworzenie Nowej Wizyty (Smart Routing)	7
4.2.2	Kaskadowe Usuwanie Pacjenta	7
4.2.3	Aktualizacja Gabinetu (Office Transaction)	8

Rozdział 1

Wprowadzenie

1.1 Cel dokumentu

Niniejszy dokument stanowi autorytatywną specyfikację techniczną systemu do zarządzania placówką medyczną. Obejmuje on zbiór wymagań funkcjonalnych, zaktualizowany model relacyjny bazy danych (uwzględniający infrastrukturę gabinetów - offices) oraz interfejs programistyczny aplikacji (API) dla backendu.

1.2 Założenia architektoniczne

Fundamentem architektury opisywanego systemu jest absolutna widoczność we mgle wszystkich procesów przetwarzania danych. System został zaprojektowany z myślą o zagwarantowaniu krystalicznej jasności operacji, co niweluje zjawisko "ukrytych błędów" lub nieautoryzowanych modyfikacji. Narzędziem realizującym to założenie jest wbudowany, kuloodporny `AuditService`, który działa analogicznie do czarnej skrzynki, utrwalając każdy stan encji (`CREATE`, `UPDATE`, `DELETE`) przed i po jej modyfikacji.

Rozdział 2

Wymagania Funkcjonalne

2.1 Wstęp

Moduł ten opisuje wymagania funkcjonalne postawione przed aplikacją ze szczególnym uwzględnieniem zarządzania zasobami ludzkimi, infrastrukturą (gabinety) oraz inteligentnym planowaniem wizyt.

2.2 Przypadki Użycia i Zachowanie Systemu

2.2.1 Zarządzanie Wizytami i Smart Calendar

- **Inteligentny Kalendarz (Smart Calendar):** Rejestracja nowej wizyty jest w pełni zautomatyzowana. System samodzielnie wylicza najbliższe wolne terminy (w 30-minutowych interwałach), wymuszając godziny pracy 8:00 - 16:00 i automatycznie omijając weekendy oraz terminy już zajęte.
- **Zarządzanie statusem i gabinetem:** Wizyty posiadają zdefiniowaną maszynę stanów (zaplanowana, zakończona, anulowana). Przypisanie gabinetu do wizyty odbywa się w tle na poziomie bazy danych (na podstawie profilu wybranego lekarza).
- **Śledzenie zmian:** Każda edycja (PUT), szybka zmiana statusu (PATCH), utworzenie (POST) oraz twarde usunięcie (DELETE) wizyty jest przechwytywane i precyzyjnie logowane w module audytu.

2.2.2 Zarządzanie Gabinetami (Offices / Rooms)

- **Relacja 1:1:** System zarządza fizycznymi gabinetami (Konsultacyjny, Zabiegowy, Diagnostyczny). Każdy lekarz może mieć przypisany maksymalnie jeden unikalny gabinet.
- **Bezpieczna edycja:** Zmiana przypisania gabinetu z poziomu administratora korzysta z mechanizmu transakcji SQL, automatycznie czyszcząc poprzednie powiązania przed nadaniem nowego właściciela pokoju.

2.2.3 Zarządzanie Lekarzami i Pacjentami

- **Kaskadowe usuwanie:** Usunięcie profilu lekarza lub pacjenta wymusza wcześniejsze usunięcie wszystkich powiązanych z nimi wizyt. Operacja objęta jest transakcją (BEGIN i COMMIT) zapobiegającą naruszeniu więzów spójności (Foreign Key Constraint). Złożony proces usuwania jest również raportowany w audycie.
- **Zabezpieczenie przed duplikacją:** PESEL posiada unikalny klucz (UNIQUE), wymuszający weryfikację na poziomie silnika bazy danych.

2.2.4 System Audytu (Blackbox) i Inicjalizacja

- **Masowe zasilanie danymi (Seeder):** Moduł `createdb.mjs` integruje FakerPL w celu transakcyjnego zasilenia bazy 50 gabinetami, 50 lekarzami, 500 pacjentami oraz 1000 logicznie spójnych wizyt (wpadających tylko w godziny pracy i unikających weekendów).

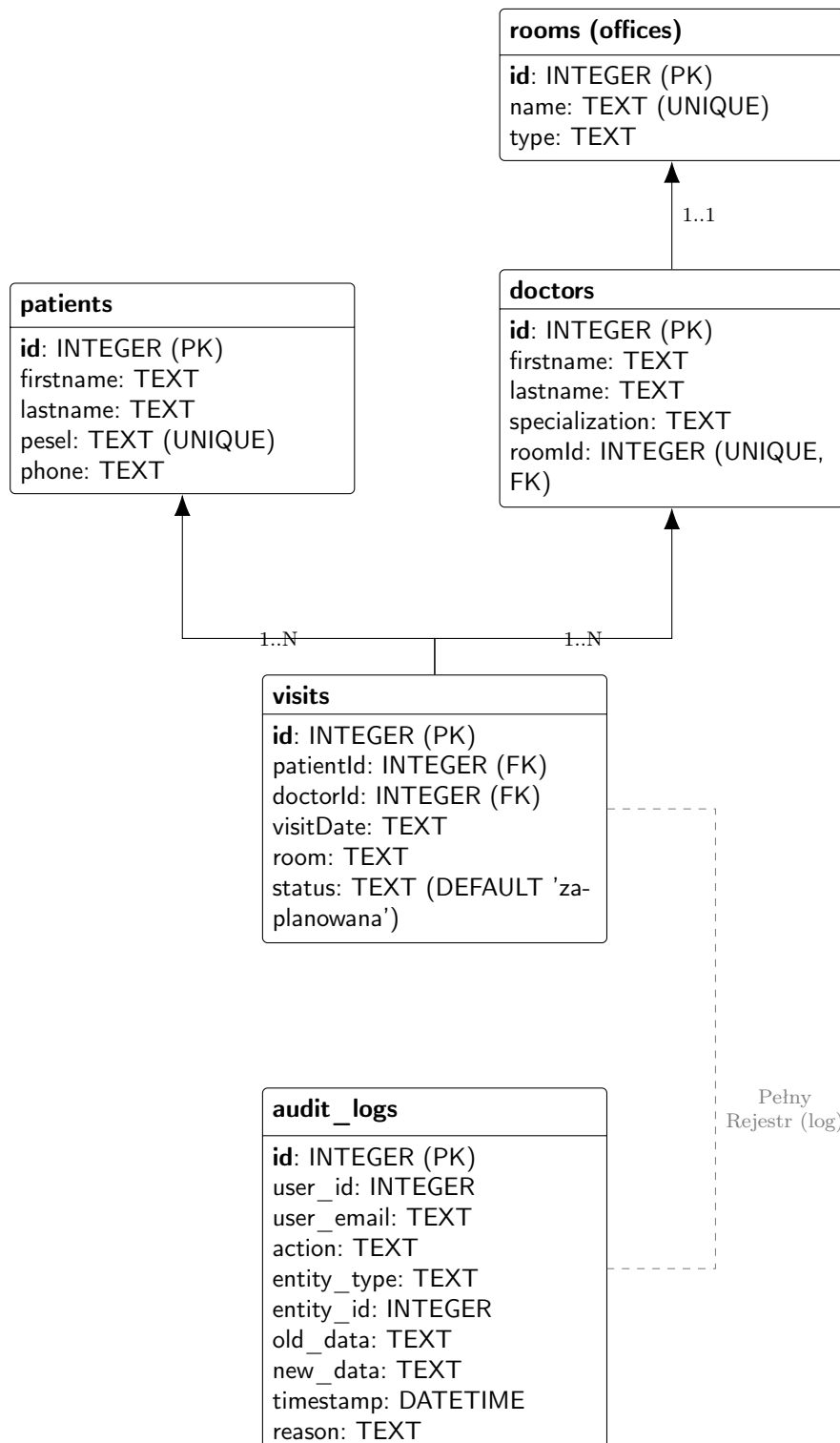
Rozdział 3

Diagram Bazy Danych

3.1 Wstęp

Silnikiem bazodanowym systemu jest relacyjna baza `SQLite`. Skrypt inicjalizacyjny zapewnia budowę struktur opartych o rygorystyczne więzy integralności (Foreign Keys = ON).

3.2 Schemat ERD (Entity-Relationship Diagram)



Rysunek 3.1: Struktura relacyjna bazy danych uwzględniająca gabinety (offices)

Rozdział 4

Specyfikacja API Backendu

4.1 Wstęp

Warstwa serwerowa została zrealizowana w środowisku Node.js z wykorzystaniem frameworka Express. Komunikacja odbywa się w standardzie REST, a ładunki użyteczne wymieniane są w formacie JSON. Wszystkie ścieżki podlegają uwierzytelnieniu ról (Role-Based Access Control).

4.2 Kluczowe Endpointy

4.2.1 Tworzenie Nowej Wizyty (Smart Routing)

Tworzy nową wizytę z automatycznym przypisaniem odpowiedniego gabinetu (weryfikacja na poziomie bazy) oraz generuje wpis audytowy.

Metoda HTTP	POST
Ścieżka	/api/visits
Wymagane role	0 (Admin), 2 (Recepcja)
Request Body	
201 Created - {"id": 1001}	
Możliwe błędy	409 Conflict (Kolizja terminów)

4.2.2 Kaskadowe Usuwanie Pacjenta

Wykonuje trwałe usunięcie zasobu pacjenta z bazy danych wraz z **kaskadowym usunięciem** jego historii chorobowej (wizyt). Czas operacji chroniony jest transakcją.

Metoda HTTP	DELETE
Ścieżka	/api/patients/:id
Wymagane role	0 (Admin)
Logika Biznesowa	Silnik egzekwuje DELETE FROM visits WHERE patientId = ?, zwalniając blokadę Foreign Key, po czym usuwa rekord pacjenta i loguje zdarzenie w Audycie wraz z kompletem pierwotnych danych.
Odpowiedź	200 OK - {"message": "Pacjent i jego wizyty zostali trwale usunięci."}

4.2.3 Aktualizacja Gabinetu (Office Transaction)

Służy do edycji nazwy gabinetu oraz bezpiecznego przełączenia powiązanego z nim lekarza bez obaw o naruszenie reguły 1:1.

Metoda HTTP	PUT
Ścieżka	/api/offices/:id
Wymagane role	0 (Admin)
Logika Biznesowa	Zawarta w instrukcjach BEGIN i COMMIT. Wyczyści przypisanie u starego lekarza (UPDATE doctors SET roomId = NULL) przed nadaniem tego samego roomId nowemu specjalście.
Request Body 200 OK - {"message": "Gabinet zaktualizowany pomyślnie"}	